

# Multimodal visual understand + visual following (Text Version)

---

## Multimodal visual understand + visual following (Text Version)

1. Course Content
2. Preparation
  - 2.1 Content Description
3. Running the Example
  - 3.1 Starting the Program
  - 3.2 Test Cases
    - 3.2.1 Example 1: "Start following red"
    - 3.2.2 Example 2: "Follow the Object in My Hand" (Depth Camera)
4. Source Code Parsing
  - 4.1 Example 1
    - 4.1.1 Nuwa Depth Camera, action\_service\_nuwa.py
    - 4.1.2 USB Camera, action\_service\_usb.py
  - 4.2 Example 2
    - 4.2.1 nuwa Depth Camera, action\_service\_nuwa.py

## 1. Course Content

---

1. Learn to use the robot's vision combined with the robot's follow-up function
2. Analyze new key source code

## 2. Preparation

---

### 2.1 Content Description

This lesson uses the Raspberry Pi 5 as an example. For Raspberry Pi and Jetson-nano , you need to open a terminal on the host computer and enter the command to enter the Docker container. Once inside the Docker container, enter the commands mentioned in this lesson in the terminal. For instructions on entering the Docker container from the host computer, refer to **[01. Robot Configuration and Operation Guide] -- [5.Enter Docker (For JETSON Nano and RPi 5)]**. For RDK X5 and Orin boards , simply open a terminal and enter the commands mentioned in this lesson.

 This example uses `model: "qwen/qwen2.5-vl-72b-instruct:free", "qwen-vl-latest"`

 The responses from the large model may not be exactly the same for the same test command and may differ slightly from the screenshots in the tutorial. To increase or decrease the diversity of the large model's responses, refer to the section on configuring the decision-making large model parameters in the **[04.AI Large Model Development] -- [5.Deploy the RAG knowledge base]** .

 It is recommended that you first try the previous visual example. This example adds voice functionality to the singleton example. The functionality is largely the same, so I will not elaborate on the debugging details of the program. The results will be detailed here! !

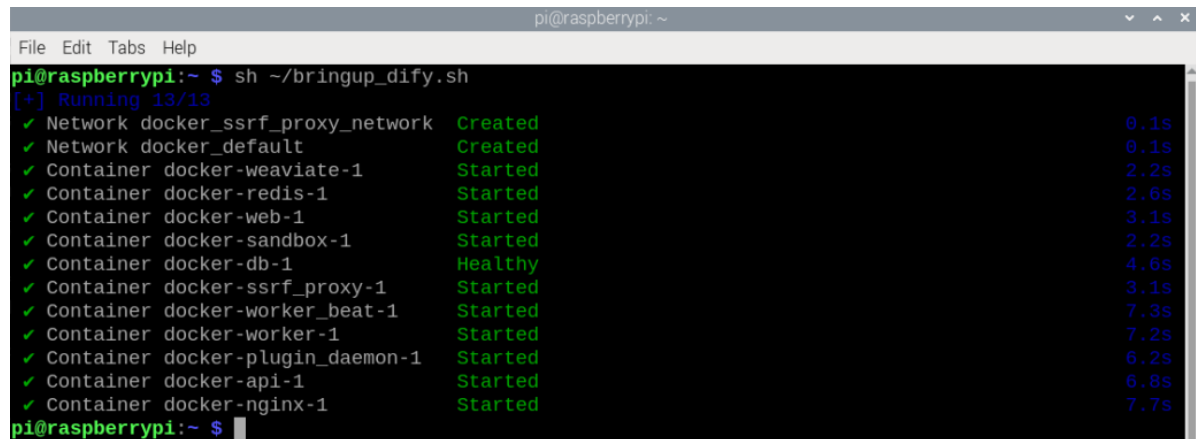
## 3. Running the Example

---

## 3.1 Starting the Program

Open the host terminal and start the Dify environment:

```
sh ~/bringup_dify.sh
```



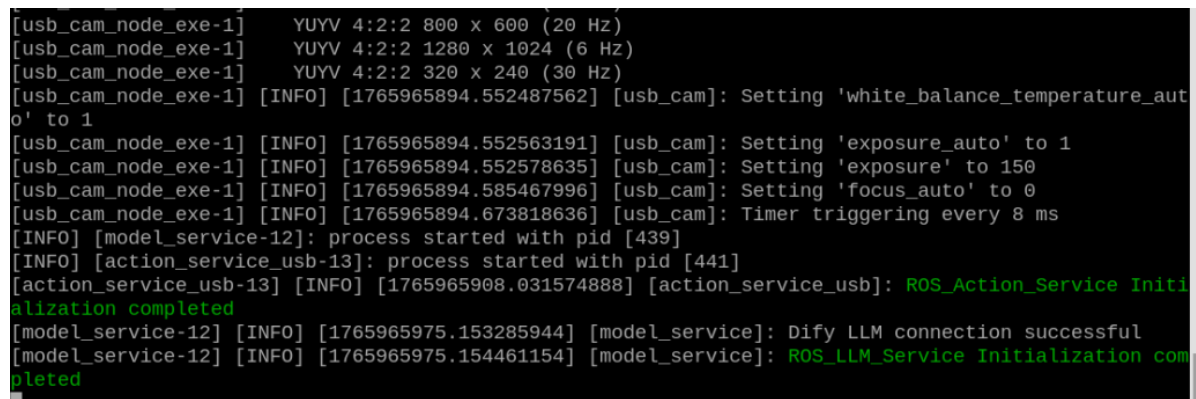
```
pi@raspberrypi: ~  
File Edit Tabs Help  
pi@raspberrypi:~$ sh ~/bringup_dify.sh  
[+] Running 13/13  
✔ Network docker_ssrf_proxy_network Created 0.1s  
✔ Network docker_default Created 0.1s  
✔ Container docker-weaviate-1 Started 2.2s  
✔ Container docker-redis-1 Started 2.6s  
✔ Container docker-web-1 Started 3.1s  
✔ Container docker-sandbox-1 Started 2.2s  
✔ Container docker-db-1 Healthy 4.6s  
✔ Container docker_ssrf_proxy-1 Started 3.1s  
✔ Container docker-worker_beat-1 Started 7.3s  
✔ Container docker-worker-1 Started 7.2s  
✔ Container docker-plugin_daemon-1 Started 6.2s  
✔ Container docker-api-1 Started 6.8s  
✔ Container docker-nginx-1 Started 7.7s  
pi@raspberrypi:~$
```

**For Raspberry Pi 5 and Jetson Nano controllers, you must first enter the Docker container. For RDKX5 and Orin board, this is not necessary.**

Open a terminal in Docker and enter the following command:

```
ros2 launch multi_brains llm_agent_control.launch.py text_chat_mode:=True
```

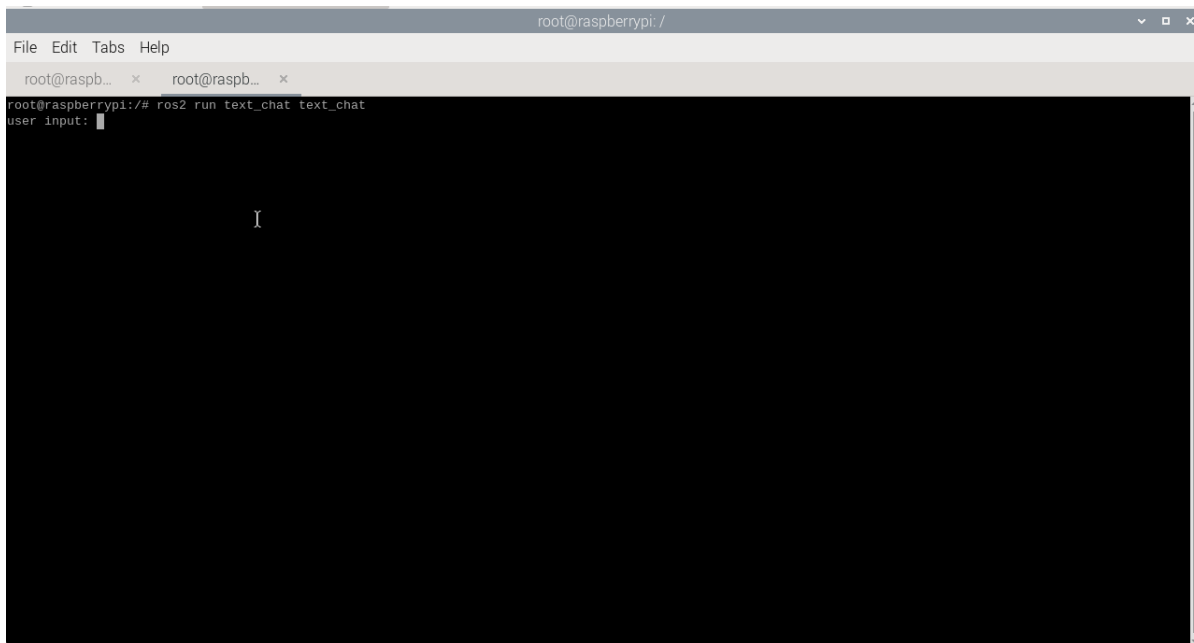
After initialization, the following content will be displayed:



```
[usb_cam_node_exe-1] YUYV 4:2:2 800 x 600 (20 Hz)  
[usb_cam_node_exe-1] YUYV 4:2:2 1280 x 1024 (6 Hz)  
[usb_cam_node_exe-1] YUYV 4:2:2 320 x 240 (30 Hz)  
[usb_cam_node_exe-1] [INFO] [1765965894.552487562] [usb_cam]: Setting 'white_balance_temperature_auto' to 1  
[usb_cam_node_exe-1] [INFO] [1765965894.552563191] [usb_cam]: Setting 'exposure_auto' to 1  
[usb_cam_node_exe-1] [INFO] [1765965894.552578635] [usb_cam]: Setting 'exposure' to 150  
[usb_cam_node_exe-1] [INFO] [1765965894.585467996] [usb_cam]: Setting 'focus_auto' to 0  
[usb_cam_node_exe-1] [INFO] [1765965894.673818636] [usb_cam]: Timer triggering every 8 ms  
[INFO] [model_service-12]: process started with pid [439]  
[INFO] [action_service_usb-13]: process started with pid [441]  
[action_service_usb-13] [INFO] [1765965908.031574888] [action_service_usb]: ROS_Action_Service Initialization completed  
[model_service-12] [INFO] [1765965975.153285944] [model_service]: Dify LLM connection successful  
[model_service-12] [INFO] [1765965975.154461154] [model_service]: ROS_LLM_Service Initialization completed
```

Open the same Docker container in multiple terminals and start it.

```
ros2 run text_chat text_chat
```



## 3.2 Test Cases

Here are some test cases for reference; users can create their own test instructions.

- Start xx tracking  
Color/Face/Object/Machine Code/QR Code/Gesture Recognition/Human Posture  
Color tracking, including red, green, blue, and yellow (requires color calibration according to the **AI Large Model Preparation** tutorial).  
Object tracking

⚠ Please do not end the text with a period or any other characters!

- ⚡ Note for gimbal servo USB camera users: **No object following!**
  - This tracking example uses the principle of near-increase-increase-difference. The size of the object being followed and the tracking effect are related to the recognized area. For program debugging, please refer to the single example.

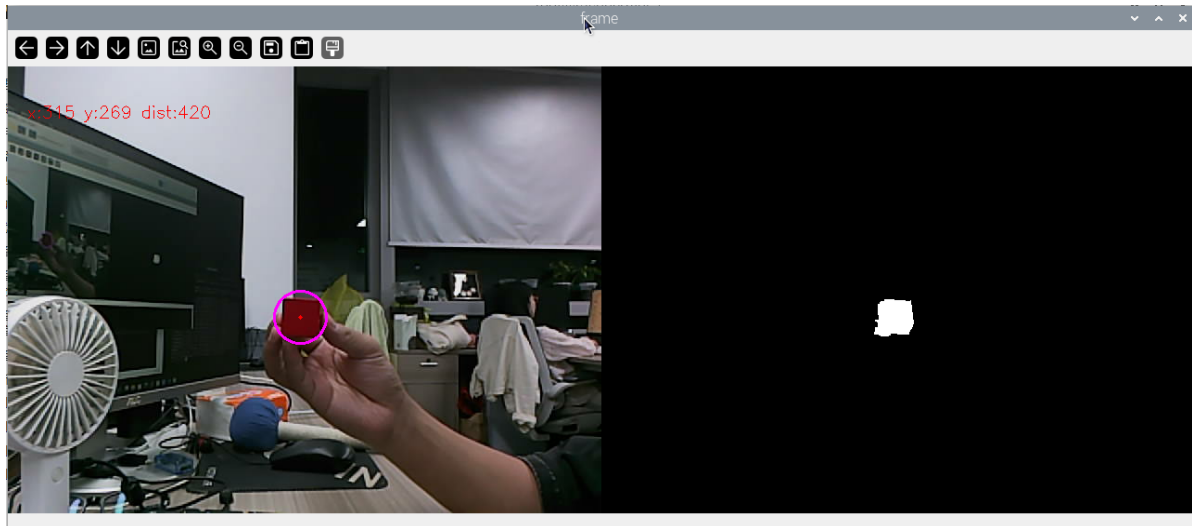
| Tracking Examples                | Recommended Reference Object Size |
|----------------------------------|-----------------------------------|
| Color Following                  | 3x3cm Building Blocks             |
| Face Following                   | Face from a Mobile Phone Image    |
| AprilTag Machine Code Following  | 3x3cm Building Blocks             |
| QR Code Tracking                 | QR Code Printed on A5 Paper       |
| Mediapipe Gesture Following      | Real-person gestures              |
| Meidpipe Human Posture Following | Real-person posture               |

### 3.2.1 Example 1: "Start following red"

Type "Start following red" in the terminal. The terminal will print the following information.

```
root@raspberrypi: /
File Edit Tabs Help
* root@raspb... x root@raspb... x
root@raspberrypi:~/# ros2 run text_chat text_chat
user input: 开始跟随红色物体 Start following red
okay👌, let me think for a moment... / [INFO] [1755602844.525915817] [text_chat_node]: 决策层AI规划:1. 调用跟随指定颜色物体函数, 参数为'red'
user input: [INFO] [1755602847.727721440] [text_chat_node]: "action": ['colcor_follow(red)'], "response": "好的, 我已经开始跟随红色物体啦, 就像一个忠诚的小跟班一样!"
```

A window titled **frame** will open on the VNC screen, displaying the current robot-viewing image.



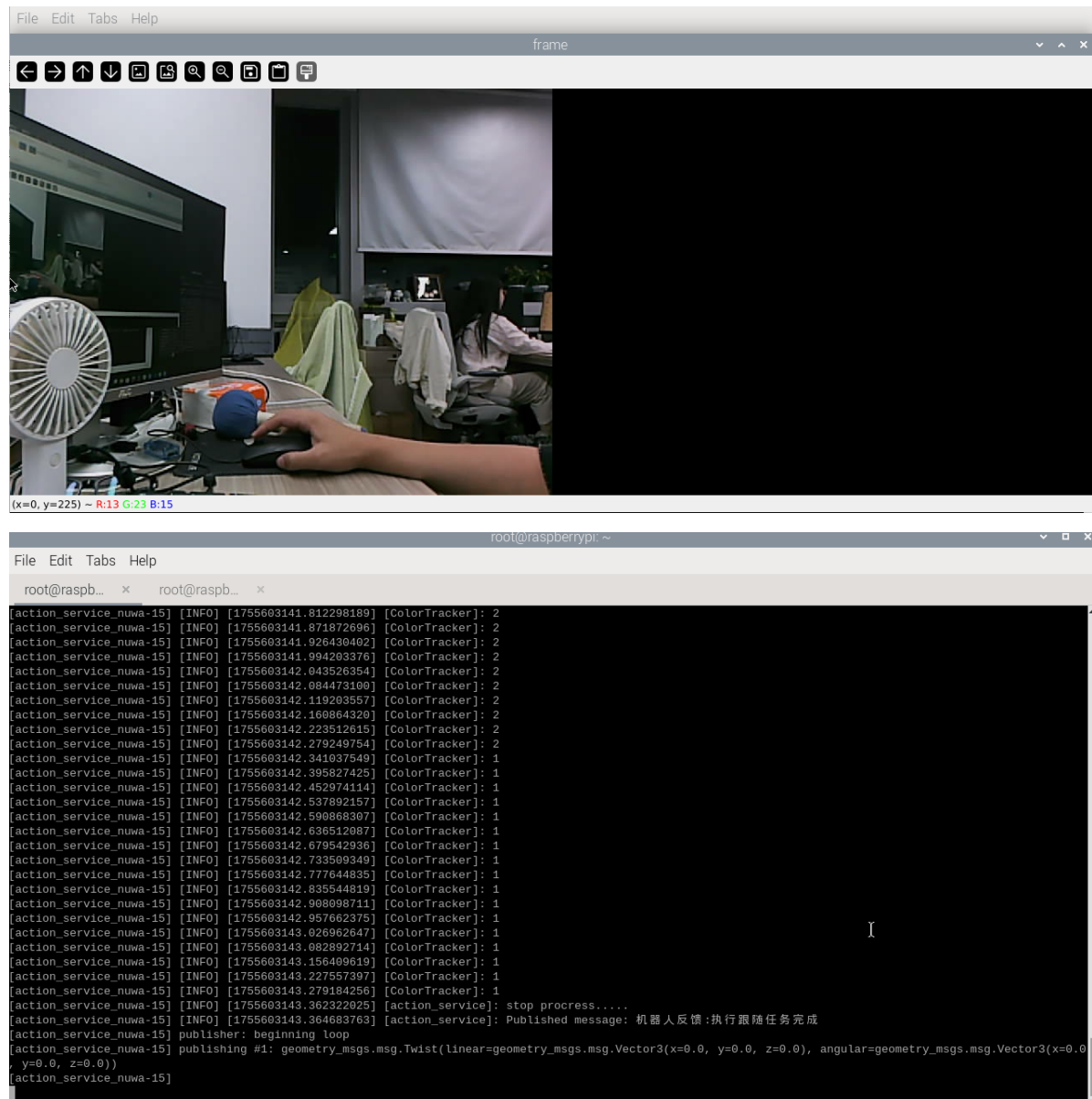
Move the object slowly, and the robot will follow.

To end the task, press **ENTER** in the terminal. Press the [Enter] key to continue the conversation, enter **[Stop Following]**, or [End Following].

```
root@raspberrypi: /
File Edit Tabs Help
* root@raspb... x root@raspb... x
root@raspberrypi:~/# ros2 run text_chat text_chat
user input: 开始跟随红色物体
okay👌, let me think for a moment... / [INFO] [1755602844.525915817] [text_chat_node]: 决策层AI规划:1. 调用跟随指定颜色物体函数, 参数为'red'
user input: [INFO] [1755602847.727721440] [text_chat_node]: "action": ['colcor_follow(red)'], "response": "好的, 我已经开始跟随红色物体啦, 就像一个忠诚的小跟班一样!"

user input: 停止跟随 Stop Following
okay👌, let me think for a moment... / [INFO] [1755602924.487956356] [text_chat_node]: "action": ['stop_follow()'], "response": "好的, 我已经停止跟随了. 还有其他任务需要我帮忙吗?"
user input: 
```

If the camera doesn't see the red object after 10 seconds, a countdown will appear on the terminal. When the countdown ends, the program will automatically stop following.



After completing a task, the robot will enter a waiting state. Pressing the **ENTER** key will continue the conversation. The command will be directly passed to the execution layer model, and all conversation history will be retained. We can enter the **"End Current Task"** command again to end the robot's current task cycle and start a new one.

```
root@raspberrypi: /
File Edit Tabs Help
*root@raspb... x root@raspb... x
root@raspberrypi: /# ros2 run text_chat text_chat
user input: 开始跟随红色物体
okay👉, let me think for a moment... / [INFO] [1755602844.525915817] [text_chat_node]: 决策层AI规划:1. 调用跟随指定颜色物体函数, 参数为'red'
user input: [INFO] [1755602847.727721440] [text_chat_node]: "action": ['color_follow(red)'], "response": "好的, 我已经开始跟随红色物体啦, 就像一个忠诚的小跟班一样!"
user input: 停止跟随
okay👉, let me think for a moment... / [INFO] [1755602924.487956356] [text_chat_node]: "action": ['stop_follow()'], "response": "好的, 我已经停止跟随了。还有其
他任务需要我帮忙吗?"
user input: 结束当前任务 End Current Task
okay👉, let me think for a moment... / [INFO] [1755604021.770832044] [text_chat_node]: "action": ['finish_dialogue()'], "response": "好的, 任务已经结束了, 有需要再叫我哦~"
user input: █
```

```
root@raspberrypi: ~
File Edit Tabs Help
root@raspb... x root@raspb... x
ze, read :48770, but framesize:732965898
[ascamera_node-1] [INFO] [1755603895.321563998] [ascamera_hp60c.camera_publisher]: 2025-08-19 19:44:55[WARN] [CameraHp60c.cpp] [784] [processRawData] wrong si
ze, read :16360, but framesize:2148958230
[ascamera_node-1] [INFO] [1755603895.322320028] [ascamera_hp60c.camera_publisher]: 2025-08-19 19:44:55[WARN] [CameraHp60c.cpp] [784] [processRawData] wrong si
ze, read :16263, but framesize:2451870244
[ascamera_node-1] [INFO] [1755603895.328776964] [ascamera_hp60c.camera_publisher]: 2025-08-19 19:44:55[WARN] [CameraHp60c.cpp] [784] [processRawData] wrong si
ze, read :77508, but framesize:2030043136
[ascamera_node-1] [INFO] [1755603895.354488856] [ascamera_hp60c.camera_publisher]: 2025-08-19 19:44:55[INFO] [CameraHp60c.cpp] [911] [setInternalParameter] m
eg info: size(640x480)
[ascamera_node-1] [INFO] [1755603895.550209437] [ascamera_hp60c.camera_publisher]: SN [ ASC60CE17000849 ]'s parameter:
[ascamera_node-1] [INFO] [1755603895.550209175] [ascamera_hp60c.camera_publisher]: irfx: 425
[ascamera_node-1] [INFO] [1755603895.550318601] [ascamera_hp60c.camera_publisher]: irfy: 425
[ascamera_node-1] [INFO] [1755603895.550332396] [ascamera_hp60c.camera_publisher]: ircx: 314.577
[ascamera_node-1] [INFO] [1755603895.550345118] [ascamera_hp60c.camera_publisher]: ircy: 237.151
[ascamera_node-1] [INFO] [1755603895.550357896] [ascamera_hp60c.camera_publisher]: rgbfx: 571
[ascamera_node-1] [INFO] [1755603895.550372377] [ascamera_hp60c.camera_publisher]: rgbfy: 571
[ascamera_node-1] [INFO] [1755603895.550388135] [ascamera_hp60c.camera_publisher]: rgbcx: 332.029
[ascamera_node-1] [INFO] [1755603895.550400468] [ascamera_hp60c.camera_publisher]: rgbcy: 235.042
[ascamera_node-1] [INFO] [1755603895.599661733] [ascamera_hp60c.camera_publisher]: publish color(rgb) info
[ascamera_node-1] [INFO] [1755603895.602285924] [ascamera_hp60c.camera_publisher]: publish tf info
[ascamera_node-1] [INFO] [1755603896.155881465] [ascamera_hp60c.camera_publisher]: 2025-08-19 19:44:56[INFO] [CameraHp60c.cpp] [1148] [streamCallback] set gai
ret 0, gain 4
[action_service_nuwa-15] [INFO] [1755603916.083887574] [ColorTracker]: Loaded 5 color profiles
[ascamera_node-1] [INFO] [1755603918.591988180] [ascamera_hp60c.camera_publisher]: publish depth info
[action_service_nuwa-15] [INFO] [1755603927.783517218] [action_service]: stop process.....
[action_service_nuwa-15] [INFO] [1755603927.790782853] [action_service]: Published message: 机器人反馈:执行跟随任务完成
[action_service_nuwa-15] publisher: beginning loop
[action_service_nuwa-15] publishing #1: geometry_msgs.msg.Twist(linear=geometry_msgs.msg.Vector3(x=0.0, y=0.0, z=0.0), angular=geometry_msgs.msg.Vector3(x=0.0
, y=0.0, z=0.0))
[action_service_nuwa-15] [INFO] [1755604021.757927962] [action_service]: Published message: finish
[model_service-14] [INFO] [1755604021.759377343] [model_service]: The current instruction cycle has ended
```

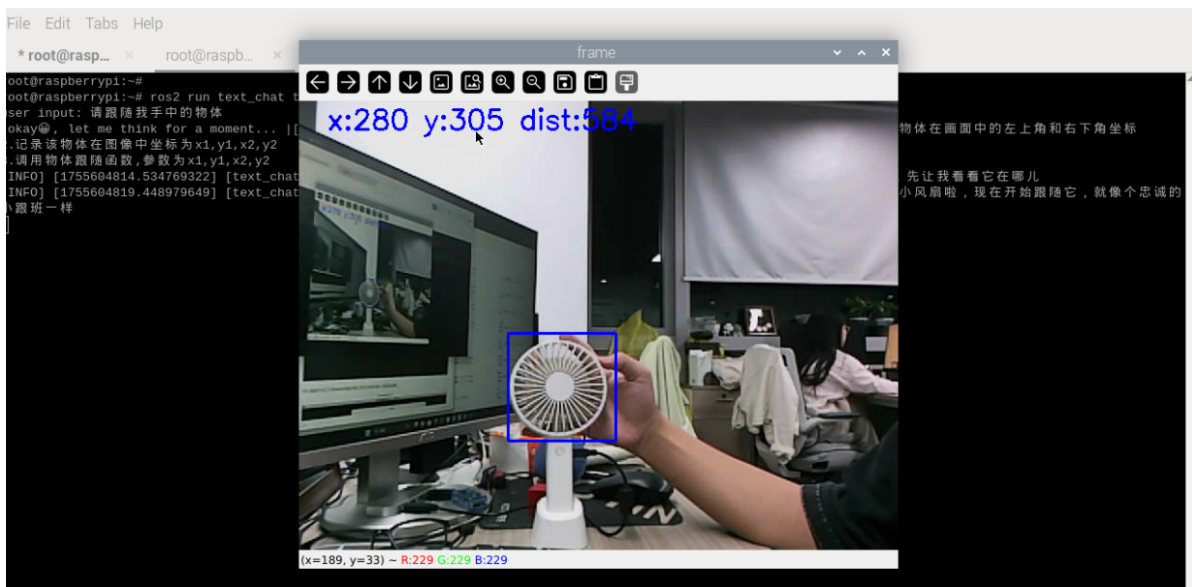
### 3.2.2 Example 2: "Follow the Object in My Hand" (Depth Camera)

⚠ The coordinates obtained by following the object in this example are entirely derived from the inference of the large AI model. Therefore, it is recommended to use a newer model for better results!

Enter "Please follow the object in my hand" in the terminal. The terminal will print the following information:

```
root@raspberrypi: ~
File Edit Tabs Help
*root@raspb... x root@raspb... x
root@raspberrypi:~# ros2 run text_chat text_chat
user input: 请跟随我手中的物体 Follow the Object in My Hand
okay, let me think for a moment... [[INFO] [1755604812.066022255] [text_chat_node]: 决策层AI规划:1.调用视觉函数获取该物体在画面中的左上角和右下角坐标
2.记录该物体在图像中坐标为x1,y1,x2,y2
3.调用物体跟随函数,参数为x1,y1,x2,y2
[INFO] [1755604814.534769322] [text_chat_node]: "action": ['seewhat()'], "response": "好的,我准备开始跟随你手中的物体啦,先让我看看它在哪儿
[INFO] [1755604819.448979649] [text_chat_node]: "action": ['KCF_follow(230, 235, 345, 350)'], "response": "我看到你手中的小风扇啦,现在开始跟随它,就像个忠诚的
小跟班一样
I
```

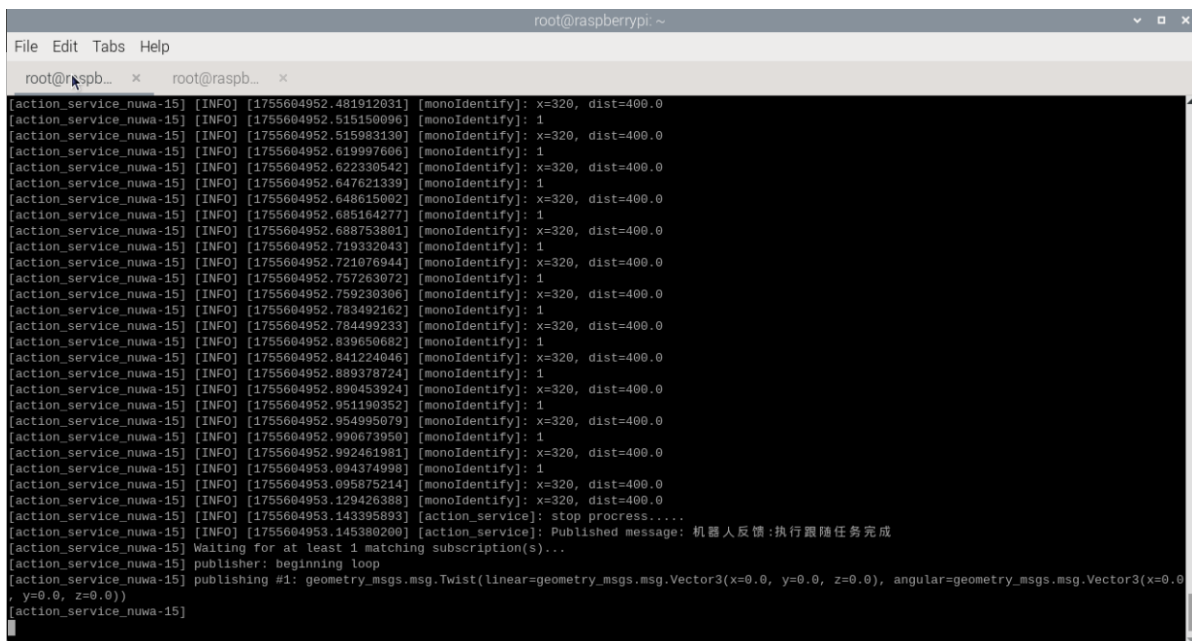
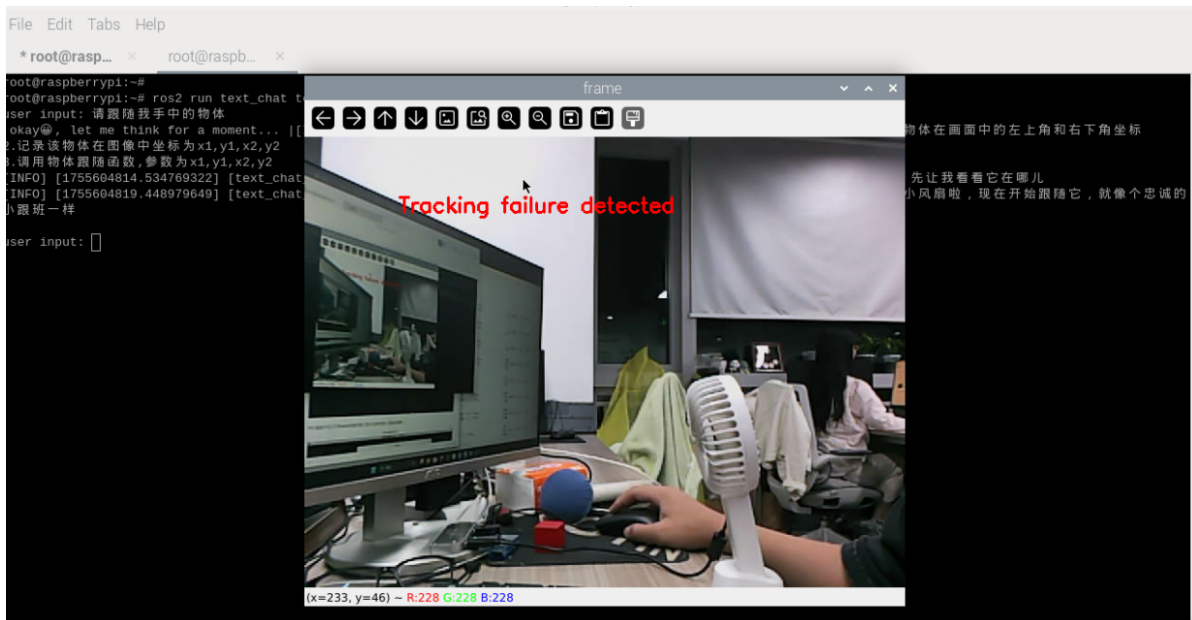
A window titled **frame** will open on the VNC screen, displaying the image from the robot's current perspective.



Move the object slowly, and the robot will follow.

If the screen If there are no targets to follow, the program will count down for 10 seconds, a 5-second countdown will be printed on the terminal, and the process will automatically end, indicating that the task has been completed.





To manually end the task, press the **ENTER** key in the terminal and continue the conversation, entering either [Stop Following] or [End Following].

After completing a task, the robot enters a waiting state. Commands at this point are directly passed to the execution layer model, and all conversation history is retained. You can enter the **"End Current Task"** command again to end the current task cycle and start a new one.



```
root@raspberrypi: ~
File Edit Tabs Help
* root@raspb... x root@raspb... x
root@raspberrypi:~#
root@raspberrypi:~# ros2 run text_chat text_chat
user input: 请跟随我手中的物体 End Current Task
okay😊, let me think for a moment... |[INFO] [1755604812.066022255] [text_chat_node]: 决策层AI规划:1.调用视觉函数获取该物体在画面中的左上角和右下角坐标
2.记录该物体在图像中坐标为x1,y1,x2,y2
3.调用物体跟随函数,参数为x1,y1,x2,y2
[INFO] [1755604814.534769322] [text_chat_node]: "action": ['seewhat()'], "response": 好的,我准备开始跟随你手中的物体啦,先让我看看它在哪儿
[INFO] [1755604819.448979649] [text_chat_node]: "action": ['KCF_follow(230, 235, 345, 350)'], "response": 我看到你手中的小风扇啦,现在开始跟随它,就像个忠诚的小跟班一样
user input: [INFO] [1755604955.807853380] [text_chat_node]: "action": ['finishtask()'], "response": 我已经成功跟随你手中的小风扇啦,任务完成得漂漂亮亮的!
user input: 结束当前任务
okay😊, let me think for a moment... |[INFO] [1755605154.087968955] [text_chat_node]: "action": ['finish_dialogue()'], "response": 好的,任务已经结束了,有需要再叫我哦~
user input: |
```

```
root@raspberrypi: ~
File Edit Tabs Help
root@raspb... x root@raspb... x
[action_service_nuwa-15] [INFO] [1755604952.515983138] [monoIdentify]: x=320, dist=400.0
[action_service_nuwa-15] [INFO] [1755604952.619997606] [monoIdentify]: 1
[action_service_nuwa-15] [INFO] [1755604952.622330542] [monoIdentify]: x=320, dist=400.0
[action_service_nuwa-15] [INFO] [1755604952.647621339] [monoIdentify]: 1
[action_service_nuwa-15] [INFO] [1755604952.648615062] [monoIdentify]: x=320, dist=400.0
[action_service_nuwa-15] [INFO] [1755604952.685164277] [monoIdentify]: 1
[action_service_nuwa-15] [INFO] [1755604952.688753861] [monoIdentify]: x=320, dist=400.0
[action_service_nuwa-15] [INFO] [1755604952.719332043] [monoIdentify]: 1
[action_service_nuwa-15] [INFO] [1755604952.721076944] [monoIdentify]: x=320, dist=400.0
[action_service_nuwa-15] [INFO] [1755604952.757263072] [monoIdentify]: 1
[action_service_nuwa-15] [INFO] [1755604952.759230306] [monoIdentify]: x=320, dist=400.0
[action_service_nuwa-15] [INFO] [1755604952.783492162] [monoIdentify]: 1
[action_service_nuwa-15] [INFO] [1755604952.784492333] [monoIdentify]: x=320, dist=400.0
[action_service_nuwa-15] [INFO] [1755604952.839650682] [monoIdentify]: 1
[action_service_nuwa-15] [INFO] [1755604952.841224046] [monoIdentify]: x=320, dist=400.0
[action_service_nuwa-15] [INFO] [1755604952.889378724] [monoIdentify]: 1
[action_service_nuwa-15] [INFO] [1755604952.890453924] [monoIdentify]: x=320, dist=400.0
[action_service_nuwa-15] [INFO] [1755604952.951190352] [monoIdentify]: 1
[action_service_nuwa-15] [INFO] [1755604952.954995079] [monoIdentify]: x=320, dist=400.0
[action_service_nuwa-15] [INFO] [1755604952.990673950] [monoIdentify]: 1
[action_service_nuwa-15] [INFO] [1755604952.992461981] [monoIdentify]: x=320, dist=400.0
[action_service_nuwa-15] [INFO] [1755604953.094374998] [monoIdentify]: 1
[action_service_nuwa-15] [INFO] [1755604953.095875214] [monoIdentify]: x=320, dist=400.0
[action_service_nuwa-15] [INFO] [1755604953.129426388] [monoIdentify]: x=320, dist=400.0
[action_service_nuwa-15] [INFO] [1755604953.143395893] [action_service]: stop proccess.....
[action_service_nuwa-15] [INFO] [1755604953.145380200] [action_service]: Published message: 机器人反馈:执行跟随任务完成
[action_service_nuwa-15] Waiting for at least 1 matching subscription(s)...
[action_service_nuwa-15] publisher: beginning loop
[action_service_nuwa-15] publishing #1: geometry_msgs.msg.Twist(linear=geometry_msgs.msg.Vector3(x=0.0, y=0.0, z=0.0), angular=geometry_msgs.msg.Vector3(x=0.0, y=0.0, z=0.0))
[action_service_nuwa-15] [INFO] [1755605154.098045803] [action_service]: Published message: finish
[model_service-14] [INFO] [1755605154.098639649] [model_service]: The current instruction cycle has ended
```

## 4. Source Code Parsing

Source code is located at:

Jetson Orin Nano:

```
#NUWA camera user
/home/jetson/yahboomcar_ros2_ws/yahboomcar_ws/src/multi_brains/multi_brains/action_service_nuwa.py
#USB camera user
/home/jetson/yahboomcar_ros2_ws/yahboomcar_ws/src/multi_brains/multi_brains/action_service_usb.py
```

```
/home/jetson/yahboomcar_ros2_ws/yahboomcar_ws/src/multi_brains/multi_brains/model_service.py
```

Jetson Nano, Raspberry Pi:

You need to first enter Docker.

```
#NUWA Camera User
/root/jetson/yahboomcar_ros2_ws/yahboomcar_ws/src/multi_brains/multi_brains/action_service_nuwa.py
#USB Camera User
/root/jetson/yahboomcar_ros2_ws/yahboomcar_ws/src/multi_brains/multi_brains/action_service_usb.py
```

```
/root/yahboomcar_ros2_ws/yahboomcar_ws/src/multi_brains/multi_brains/model_service.py
```

RDkX5:

```
#NUWA Camera User
/home/sunrise/yahboomcar_ros2_ws/yahboomcar_ws/src/multi_brains/multi_brains/action_service_nuwa.py
#USB Camera User
/home/sunrise/yahboomcar_ros2_ws/yahboomcar_ws/src/multi_brains/multi_brains/action_service_usb.py
```

```
/home/sunrise/yahboomcar_ros2_ws/yahboomcar_ws/src/multi_brains/multi_brains/model_service.py
```

## 4.1 Example 1

### 4.1.1 Nuwa Depth Camera, action\_service\_nuwa.py

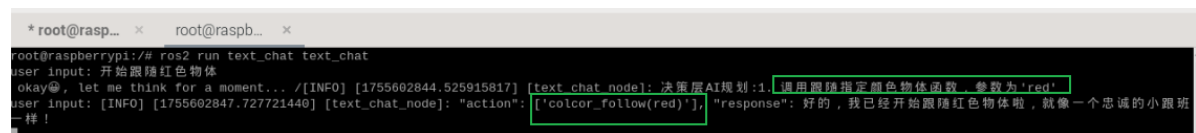
Example 1 uses the **seewhat**, **color\_follow**, and **stop\_follow()** methods in the **CustomActionServer** class.

The **seewhat** function primarily retrieves the color image from the depth camera.

The **color\_follow(self, color)** function performs color following.

The **stop\_follow()** function issues a stop command to follow.

This section focuses on the **color\_follow(self, color)** function, which requires a color parameter, which can be 'red', 'green', 'blue', or 'yellow'.



```
* root@raspb... x root@raspb... x
root@raspberrypi:~/ros2_run_text_chat text_chat
user input: 开始跟随红色物体
okay, let me think for a moment... [INFO] [1755602844.525915817] [text_chat_node]: 决策层AI规划:1 [通用跟随指定颜色物体函数 参数为'red']
user input: [INFO] [1755602847.727721440] [text_chat_node]: "action": ["colcor_follow(red)"], "response": "好的，我已经开始跟随红色物体啦，就像一个忠诚的小跟班一样！"
```

```
#Start the color line patrol subprocess
process_1 = subprocess.Popen(['ros2', 'run', 'yahboomcar_voice_ctrl_depth',
'voice_colorTracker', '--ros-args', '-p', f'color:={target_color}'])
```

The source code path of the subprocess in the program,

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_voice_ctrl_depth/yahboomcar_voice_ctrl_depth/follow/voice_colorTracker.py
```

```
def colcor_follow(self,color):
    self.colcor_follow_future = Future() #Reset Future object
    color = color.strip('"\'') # Remove single and double quotes
```

```

if color == 'red':
    target_color = float(1)
elif color == 'green':
    target_color = float(2)
elif color == 'blue':
    target_color = float(3)
elif color == 'yellow':
    target_color = float(4)
else:
    self.get_logger().info('color_sort:error')
    return

#Start the color line patrol subprocess
process_1 = subprocess.Popen(['ros2', 'run', 'yahboomcar_voice_ctrl_depth',
'voice_colorTracker', '--ros-args', '-p', f'colcor:={target_color}'])

#Waiting to stop following instructions
while not self.colcor_follow_future.done():
    if self.interrupt_flag:
        break
    time.sleep(0.1)

self.kill_process_tree(process_1.pid)
self.cancel()

```

When the main model receives the user input of the "Stop Following" or "End Following" command,

or if the target being followed is lost for more than 10 seconds,

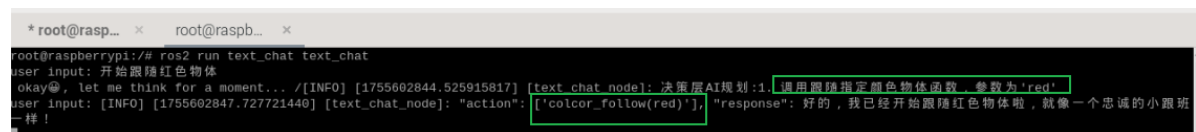
the **stop\_follow** method is called, sending the future.done signal. The `while not self.colcor_follow_future.done()` block in the **colcor\_follow** function then exits. The **kill\_process\_tree** method is then called to recursively kill the child process tree. Finally, the status of the execution action is reported to the main model at the execution layer.

#### 4.1.2 USB Camera, action\_service\_usb.py

Example 1 uses the **seewhat**, **colorFollow**, and **stop\_track()** methods in the **CustomActionServer** class.

- The **seewhat** function primarily retrieves the camera's color image.
- The **colorFollow(self, color)** function performs color tracking.
- The **stop\_track()** function issues a stop tracking command.

This section focuses on the **colorFollow(self, color)** function, which requires a color parameter (red, green, blue, and yellow).



```

*root@raspb... x root@raspb... x
root@raspberrypi:~/# ros2 run text_chat text_chat
user input: 开始跟随红色物体
okay~, let me think for a moment... /INFO] [1755602844.525915817] [text_chat_model]: 决策层AI规划:1. 调用跟随指定颜色物体函数 参数为'red'
user input: [INFO] [1755602847.727721440] [text_chat_node]: "action": ["colcor_follow(red)"], "response": "好的，我已经开始跟随红色物体啦，就像一个小跟班一样！"

```

```

#Start the color line patrol subprocess
process_1 = subprocess.Popen(['ros2', 'run', 'yahboomcar_voice_ctrl',
'colorFollow', '--ros-args', '-p', f'target_color:={target_color}'])

```

The source code path of the subprocess in the program,

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_voice_ctrl/yahboomcar_voice_ctrl/  
colorFollow.py
```

```
def colorFollow(self,color):  
    try:  
        self.colorFollow_future = Future()  
        color = color.strip("\")  
        if color == 'red':  
            target_color = int(1)  
        elif color == 'green':  
            target_color = int(2)  
        elif color == 'blue':  
            target_color = int(3)  
        elif color == 'yellow':  
            target_color = int(4)  
        else:  
            target_color = int(1)  
        process_1 = subprocess.Popen(['ros2', 'run', 'yahboomcar_voice_ctrl',  
'colorFollow', '--ros-args', '-p', f'target_color:={target_color}'])  
        while not self.colorFollow_future.done():  
            if self.interrupt_flag:  
                break  
            time.sleep(0.1)  
        self.get_logger().info(f'killed process_pid')  
        self.kill_process_tree(process_1.pid)  
        self.cancel()  
    except:  
        self.get_logger().error('colorFollow Startup failure')  
    return
```

When the main model receives the user input of the "Stop Following" or "End Following" command,

or if the tracking target is lost for more than 10 seconds,

the **stop\_track** method is called, sending the future.done signal. The `while not self.colorFollow_future.done()` block in the **colorFollow** function then exits. The **kill\_process\_tree** method is then called to recursively kill the child process tree. Finally, the status of the action execution is reported to the main model at the execution layer.

## 4.2 Example 2

### 4.2.1 nuwa Depth Camera, action\_service\_nuwa.py

Example 2 uses the **seewhat**, **KCF\_follow**, and **stop\_track** methods in the **CustomActionServer** class.

- The **seewhat** function primarily obtains the camera's color image.
- The **KCF\_follow(self,x1,y1,x2,y2)** function performs object tracking. - **stop\_track()** issues a stop command to follow the function.

The **seewhat** function primarily retrieves the camera's color image. The **KCF\_follow(self,x1,y1,x2,y2)** function takes as parameters the coordinates of the upper-left and lower-right vertices of the object's bounding box (the upper-left corner of the image is the pixel origin). For example, the coordinates of the outer bounding box of the identified green square in

Example 2 can be found from the large model's response: the upper-left corner is (230, 345) and the lower-right corner is (235, 350).

```
File Edit Tabs Help
* root@raspb... x root@raspb... x
root@raspberrypi:~#
root@raspberrypi:~# ros2 run text_chat text_chat
ser input: 请跟随我手中的物体
okay! let me think for a moment... [[INFO] [1755604812.066022255] [text_chat_node]: 决策层AI规划:1.调用视觉函数获取该物体在画面中的左上角和右下角坐标
.记录该物体在图像中坐标为x1,y1,x2,y2
.调用物体跟踪函数,参数为x1,y1,x2,y2
[INFO] [1755604814.534769322] [text_chat_node]: "action": ['seewhat()'], "response": "好的,我准备开始跟随你手中的物体啦,先让我看看它在哪儿
[INFO] [1755604819.448979649] [text_chat_node]: "action": ['KCF_follow(230, 235, 345, 350)'], "response": "我看到你手中的小风扇啦,现在开始跟随它,就像个忠诚的
小跟班一样"
```

#Start the object tracking subprocess

```
process_1 = subprocess.Popen(['ros2', 'run', 'yahboomcar_voice_ctrl_depth',
'voice_KCF_Tracker', '--ros-args', '-p', f'x1:={x1}', '-p', f'y1:={y1}', '-p', f'x2:={
x2}', '-p', f'y2:={y2}'])
```

The startup program source code path is:

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_voice_ctrl_depth/yahboomcar_voice
_ctrl_depth/kcf/voice_KCF_Tracker.py
```

```
def KCF_follow(self,x1,y1,x2,y2):
    self.KCF_follow_future = Future() #Reset Future object
    x1 = int(x1)
    y1 = int(y1)
    x2 = int(x2)
    y2 = int(y2)

    process_1 = subprocess.Popen(['ros2', 'run', 'yahboomcar_voice_ctrl_depth',
'voice_KCF_Tracker', '--ros-args', '-p', f'x1:={x1}', '-p', f'y1:={y1}', '-p', f'x2:={
x2}', '-p', f'y2:={y2}'])
    # time.sleep(1.0)#sleep for 2 seconds to wait for the thread to stabilize

    while not self.KCF_follow_future.done():
        if self.interrupt_flag:
            break
        time.sleep(0.1)

    self.kill_process_tree(process_1.pid)
    self.cancel()
```

When the main model receives the user input of the "Stop Following" or "End Following" command,

or if the target is lost for more than 10 seconds,

the **stop\_follow** method is called, sending the future.done signal. The `while not self.KCF_follow_future.done()` block in the **KCF\_follow** function is then exited. The **kill\_process\_tree** method is then called to recursively kill the child process tree. Finally, the status of the execution action is reported to the main model at the execution layer.

